

CS228 Logic for Computer Science

Exercise Problem Set 5

Note on the DPLL Algorithm: The DPLL algorithm, used in this tutorial, consists of the following steps, which are done before each decision step. In Lecture 9, both of these were grouped under a single step called unit propagation (UP); in Lecture 10 (CDCL), the pure literal check is dropped altogether in practice, as it is too expensive. Here, we separate them into two explicit steps for clarity.

1. **Unit propagation (UP):** If a clause contains only a single unassigned literal (unit clause), the variable is assigned the necessary value to make this literal true.
2. **Pure literal elimination (PLE):** If a propositional variable occurs with only one polarity in the formula, it is called pure. The variable is assigned the necessary value to make all the clauses containing this literal true.

Part 1

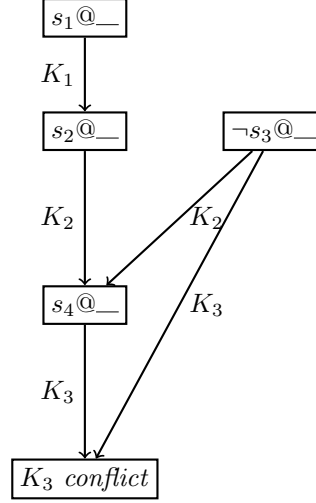
1. Let $\varphi = \varphi_1 \wedge \cdots \wedge \varphi_5$, where

$$\begin{aligned}\varphi_1 &:= x_1 \rightarrow (x_2 \vee x_3) \\ \varphi_2 &:= x_2 \rightarrow (\neg x_3 \vee \neg x_4) \\ \varphi_3 &:= x_3 \rightarrow (\neg x_2 \vee x_4) \\ \varphi_4 &:= x_4 \rightarrow \neg(x_1 \vee x_2) \\ \varphi_5 &:= \neg x_4 \rightarrow \neg(\neg x_3 \vee x_3)\end{aligned}$$

- (a) Convert each φ_i to NNF.
 - (b) Convert each φ_i to CNF (by any technique).
 - (c) Using the CNF form, write the clauses and then check satisfiability using DPLL.
2. Consider a CNF formula containing (among others) the clauses

$$\begin{aligned}K_1 &= (\neg s_1 \vee s_2) \\ K_2 &= (\neg s_2 \vee s_3 \vee s_4) \\ K_3 &= (\neg s_4 \vee s_3)\end{aligned}$$

A run of DPLL on this formula reaches the (partial) implication graph below, ending in a conflict. Each node is labelled with its literal, with a blank left for its decision level; edges are labelled with the clause responsible for the propagation.



- (a) Fill in the blank at each node with its decision level, and state whether the node is a decision literal or a propagated literal.
 - (b) Write down the conflict clause.
3. Consider the formula with the following clauses:

$$\begin{aligned}
 C_1 &= (\neg p_1 \vee \neg p_2) \\
 C_2 &= (\neg p_1 \vee p_3) \\
 C_3 &= (\neg p_3 \vee \neg p_4) \\
 C_4 &= (p_2 \vee p_4 \vee p_5) \\
 C_5 &= (\neg p_5 \vee \neg p_6 \vee \neg p_7) \\
 C_6 &= (\neg p_6 \vee p_7 \vee \neg p_8) \\
 C_7 &= (p_8 \vee \neg p_9) \\
 C_8 &= (p_8 \vee p_9 \vee \neg p_1)
 \end{aligned}$$
 - (a) Simulate a run of the DPLL algorithm on the above formula. Whenever you need to make a decision, use the decision strategy ‘assign true to the currently unassigned variable of least index’. (For example if p_2, p_4 and p_5 are unassigned, in the decision step, you should choose p_2 and assign it to true.)
 - (b) Draw the implication graph, clearly showing your decision and propagation nodes.
 - (c) Since clause learning adds the disjunction of the negations of all the decisions that caused the conflict, write down the conflict clause.
 - (d) Simulate a run of the DPLL algorithm on the above formula with the conflict clause added to the original formula, i.e., run the CDCL algorithm.
4. (We have done this as an exercise in class. The goal is to check if you understood and can replicate the formal arguments!) Let F be a CNF formula, π a partial assignment reached by a run of DPLL on F that ends in a conflict, and let C be the resulting conflict clause (the disjunction of the negations of the decisions that caused the conflict).
 - (a) Show that $F \models C$. (*Hint*: show that $F \wedge \neg C$ is unsatisfiable.)
 - (b) Show that the partial assignment π that led to the conflict will never be tried again once C is added to F and DPLL backtracks.

- (c) Can you also show $F \vdash C$ using resolution? More formally: give a resolution derivation of C from F in which each resolution step corresponds to exactly one propagation edge of the implication graph.

Part 2

5. Let n be the number of propositional variables. What are the best-case and worst-case running times of the DPLL algorithm, measured by the number of decisions it makes, as a function of n . For each scenario, give a family of formulas $\{F_n\}_{n \geq 1}$ that achieves it, with justification.
6. Recall from question 4 of Problem Sheet 4 that a formula is said to be in 2-CNF, when every clause contains at most two literals. Assume that a 2-CNF formula is given as an input to DPLL. Prove that every clause that is learnt during its execution is either empty or a singleton.

For the next two questions, recall from question 5 of Problem Sheet 4: a clause is called a **Horn clause** if it contains at most one positive literal. A CNF formula, all of whose clauses are Horn clauses, is called a **Horn formula**. Let **HornSAT** denote the problem of deciding whether a Horn formula is satisfiable or not.

7. Consider the version of DPLL that always chooses to assign 0 to a decision variable before assigning 1 (if needed). Then, show that, when a Horn formula encoded in CNF is given to this algorithm as input, the algorithm will never need to *backtrack*.
8. Let us now try to solve HornSAT using DPLL. We can convert each Horn clause into a CNF clause by simply rewriting $(a \rightarrow b)$ as $(\neg a \vee b)$, which preserves the semantics of the formula. The resultant formula is in CNF.

Suppose our version of DPLL always assigns 1 to a decision variable before assigning 0 (if needed). How many backtrackings are needed if we run this version of DPLL on the (CNF-ised version of) the following Horn formulae, assuming DPLL always chooses the unassigned variable with the smallest subscript when choosing a decision variable?

(a)

$$\left(\left(\bigwedge_{i=0}^{n-1} x_i \right) \rightarrow x_n \right) \wedge \bigwedge_{i=0}^{n-1} (x_n \rightarrow x_i) \wedge \left(\left(\bigwedge_{i=0}^n x_i \right) \rightarrow \perp \right)$$

(b)

$$\bigwedge_{i=0}^{n-1} ((x_i \rightarrow x_{n+i}) \wedge (x_{n+i} \rightarrow x_i) \wedge (x_i \wedge x_{n+i} \rightarrow \perp))$$